

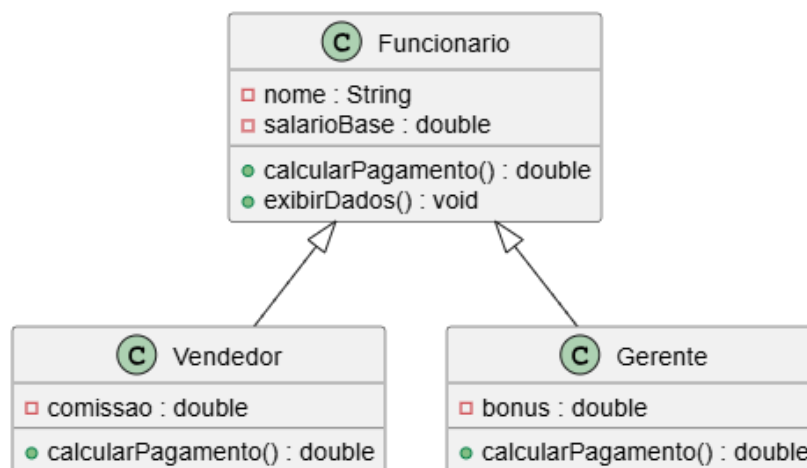
Exercício 7: Herança e Polimorfismo em Java

Contexto:

Você foi contratado por uma empresa de tecnologia para criar um **sistema de controle de funcionários**.

O sistema precisa permitir cadastrar diferentes tipos de funcionários e calcular o **pagamento mensal** de cada um, considerando regras específicas.

O **Diagrama de Classes** abaixo representa a estrutura do sistema proposta pela empresa:



1. Crie um projeto em Java e desenvolva as classes como proposto no Diagrama de Classes.

- a. **Classe Funcionário**

- i. **Atributos:**

1. `nome`: nome do funcionário.
2. `salarioBase`: valor fixo mensal.

- ii. **Métodos:**

1. `calcularPagamento()`: retorna o valor do salário base.
2. `exibirDados()`: exibe nome e valor total do pagamento

- b. **Classe Vendedor**

- i. **Atributo:**

1. `comissao`: valor variável ganho por vendas.

- ii. **Método sobrescrito:**

1. `calcularPagamento()`: retorna a soma do `salarioBase` + `comissao`.

- c. **Classe Gerente**

- i. **Atributo:**

1. bonus: valor adicional mensal por desempenho.
- ii. **Método sobrescrito:**
 1. calcularPagamento(): retorna a soma do salarioBase + bonus.

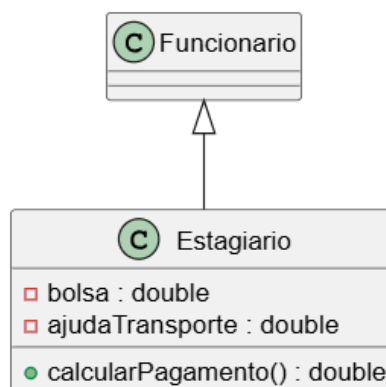
d. Adicione (de forma automatizada) os métodos construtor, *getters* e *setters* e implemente o encapsulamento.

2. Polimorfismo em ação.

- a. No método *main()* crie uma lista de funcionários (`ArrayList<Funcionario>`) contendo objetos de diferentes tipos (`Funcionario`, `Vendedor`, `Gerente`).
- b. Percorra essa lista com um loop *for-each* e chame o método *exibirDados()* para cada elemento.

Lembre-se: mesmo sendo uma lista de `Funcionario`, o método chamado será o da classe correspondente – isso é polimorfismo!

3. Vamos ampliar o contexto anterior;



- a. O pagamento deve ser `bolsa + ajudaTransporte`.
- b. Modifique o método *exibirDados()* para também mostrar o tipo do funcionário (ex: "Gerente", "Vendedor", etc.).
- c. Adicione um atributo opcional `departamento` à classe `Funcionario`.

4. Parte conceitual. Responda em seu caderno ou documento de texto:

- a. Desenhe o Diagrama de Classes completo e atualizado (Sugestão de ferramenta: PlantUML).
- b. O que é **herança** e como ela evita duplicação de código?
- c. O que significa **sobrescrever (override)** um método?
- d. Explique, com suas palavras, o que acontece quando chamamos `calcularPagamento()` em um array de `Funcionario` que contém objetos de diferentes tipos.
- e. Dê um exemplo prático (fora do contexto da empresa) onde o **polimorfismo** seria útil.